

The Definitive Guide to Power Query M

MODULE

8

API Integration and Web Connectors

Connect Power Query M to any REST API: Web.Contents, Json.Document, XML parsing, paginated endpoints, OAuth 2.0, API keys, query parameters, dynamic URL construction, and rate-limit handling.

Build production-grade connectors that refresh reliably in Power BI Service.

Module 8: API Integration and Web Connectors

Module Contents

■ Module Overview

■ Lesson 1: Web.Contents — The Core HTTP Function

- Basic GET Requests
- Query and Header Options
- RelativePath and the IsRetry Pattern
- Timeout, ManualStatusHandling, and ExcludedFromCacheKey

■ Lesson 2: Parsing API Responses

- Json.Document — Parsing JSON
- Xml.Document and Xml.Tables — Parsing XML
- Navigating Nested JSON — Records, Lists, and Tables
- Normalising Deeply Nested Responses

■ Lesson 3: Authentication Patterns

- Anonymous and API Key Authentication
- Basic Authentication
- OAuth 2.0 — Bearer Tokens
- Storing Credentials Securely in Power BI Service

■ Lesson 4: Pagination

- Offset and Limit Pagination
- Page-Number Pagination
- Cursor and Next-Link Pagination
- List.Generate — The Universal Pagination Engine

■ Lesson 5: Dynamic URL and Parameter Construction

- Building URLs with Text.Combine and Uri.BuildQueryString
- Date-Driven API Requests
- Multi-Segment Endpoint Construction
- Driving Requests from a Parameter Table

■ Lesson 6: Robustness and Error Handling

- HTTP Status Code Handling
- Retry Logic for Rate-Limited APIs
- Response Validation and Defensive Parsing
- Incremental Load Patterns

■ Lesson 7: Real-World API Patterns for PL-300

- Pattern 1: REST API with Pagination and Auth
- Pattern 2: Multi-Endpoint Connector from Config
- Pattern 3: Delta / Incremental Refresh via API
- Pattern 4: Parallel Requests with List.Transform

■ Demonstration 1: GitHub REST API — Paginated Issues

■ Demonstration 2: Multi-Endpoint CRM Connector

■ Lab 8: API Integration and Web Connectors

- Exercise 1: Basic REST Requests and JSON Parsing
- Exercise 2: Authentication and Secure Refresh

- Exercise 3: Pagination Patterns
- Exercise 4: Dynamic URLs and Production Connector

■ Module Review and Takeaways

Module Overview

The ability to connect directly to REST APIs from Power Query M is one of the most commercially valuable skills in the Power BI ecosystem. The vast majority of modern SaaS platforms — Salesforce, HubSpot, Shopify, Stripe, Google Analytics, Azure DevOps, GitHub, ServiceNow — expose REST APIs that can be consumed directly in M without a third-party connector. This module teaches every technique required.

All API work in M is built on a single function: `Web.Contents`. Understanding its options record in depth — how query parameters, headers, relative paths, and cache keys interact — is the foundation of every connector. On top of that, pagination, authentication, error handling, and dynamic URL construction complete the production-grade picture.

NOTE

This module builds directly on Module 7 (Control Flow). Pagination patterns use `List.Generate` and recursive functions. All error handling uses `try...otherwise` and `Error.Record`. Complete Modules 1–7 before this module.

Before You Begin

- Ensure Power BI Desktop is configured to allow anonymous web requests in Trust settings.
- Have a free GitHub account — labs use the public GitHub REST API (no API key required for read-only).
- Download lab support file: `APIConfig.xlsx` (endpoint config table used in exercises).
- Install a REST client (Postman, Insomnia, or browser DevTools) to inspect raw API responses first.

LESSON 1

Web.Contents — The Core HTTP Function

After completing this lesson, you will be able to:

- Make basic GET requests with Web.Contents.
- Pass query string parameters, custom headers, and request body via the options record.
- Use RelativePath to build URL segments while keeping the base URL cacheable.
- Apply ManualStatusHandling to inspect HTTP error codes without crashing the query.

Syntax	Web.Contents(url, optional options as record) as binary	
Parameter	Type	Description
url	text	The base URL. May be a complete URL or a base when using RelativePath.
options	record	Optional. Controls: Query, Headers, RelativePath, Content, Timeout, IsRetry, ManualStatusHandling, ExcludedFromCacheKey.
Returns	Returns the binary content of the HTTP response. Pass to Json.Document, Xml.Document, or Csv.Document to parse.	
Example	Web.Contents("https://api.github.com/repos/microsoft/vscode/issues")	
Result	Binary HTTP response body	

NOTE

Web.Contents always returns binary. You must pipe the result to a parser:
Json.Document(Web.Contents(...)) for JSON, or Xml.Document(Web.Contents(...)) for XML.

The options record controls every aspect of the HTTP request. The most important fields are:

```
// Full options record – all fields are optional
Web.Contents("https://api.example.com/data", [
// Query string parameters – appended as ?key=value&key2;=value2
Query = [ page = "1", per_page = "100", status = "active" ],

// HTTP headers – authentication, content type, API version
Headers = [
#"Authorization" = "Bearer " & ApiToken,
#"Accept" = "application/json",
#"X-API-Version" = "2024-01"
],

// Appended to base URL – allows base URL to remain static for caching
RelativePath = "users/12345/orders",

// POST body – omit for GET requests
Content = Text.ToBinary( Json.FromValue([ status="active", limit=100 ]) ),

// Timeout in seconds (default varies by connector)
Timeout = #duration(0,0,30,0),
```

```
// Tell M not to treat HTTP 4xx/5xx as M-level errors – handle them yourself
ManualStatusHandling = {400, 401, 403, 404, 429, 500, 503},

// Keys excluded from the query cache – dynamic timestamps, etc.
ExcludedFromCacheKey = {"page"}
})
```

WARNING

Never concatenate user-controlled values directly into the URL string. Always use the Query record for parameters — it handles URL encoding automatically. Concatenation is a security risk and breaks on values containing spaces or special characters.

RelativePath and Caching

Power BI Service caches Web.Contents results based on the base URL. If you put dynamic values (page numbers, dates) directly in the URL string, every distinct URL is treated as a separate cached entry — or worse, refresh validation fails entirely. The correct pattern separates the static base from dynamic parts:

```
// WRONG – dynamic value in URL string breaks caching and refresh validation
Web.Contents("https://api.example.com/data?page=" & Text.From(PageNum))

// CORRECT – static base URL, dynamic values in RelativePath or Query
Web.Contents(
  "https://api.example.com", // static – cached by Power BI Service
  [
    RelativePath = "data",
    Query = [ page = Text.From(PageNum), per_page = "100" ]
  ]
)

// CORRECT – RelativePath for path segments, Query for parameters
Web.Contents(
  "https://api.example.com",
  [
    RelativePath = "users/" & UserID & "/orders",
    Query = [ status = "completed", since = DateStr ]
  ]
)
```

ManualStatusHandling and IsRetry

```
// Inspect HTTP status code without crashing
let
  Response = Web.Contents("https://api.example.com/resource/99999",
    [ ManualStatusHandling = {404, 429, 500} ]),
  Status = Value.Metadata(Response)[Response.Status],
  Result =
    if Status = 200 then Json.Document(Response)
    else if Status = 404 then null
    else if Status = 429 then error Error.Record("RateLimit",
      "API rate limit exceeded. Retry after delay.", [Status=Status])
    else error Error.Record("HttpError",
      "Unexpected status: " & Text.From(Status), [Status=Status])
  in
  Result

// IsRetry – tells the connector this is a retry attempt (bypasses cache)
```

```
Web.Contents("https://api.example.com/data",  
[ IsRetry = true, Query = [ ts = Text.From(DateTime.LocalNow()) ] ])
```

LESSON 2

Parsing API Responses

After completing this lesson, you will be able to:

- Parse JSON API responses with `Json.Document`.
- Parse XML responses with `Xml.Document` and `Xml.Tables`.
- Navigate nested JSON structures — records within lists within records.
- Normalise deeply nested API responses into flat tables for the data model.

Syntax	<code>Json.Document(jsonText as any, optional encoding as number) as any</code>	
Parameter	Type	Description
<code>jsonText</code>	binary or text	The JSON content — typically the result of <code>Web.Contents</code> .
<code>encoding</code>	number	Optional. Text encoding. Default: UTF-8.
Returns	Parses a JSON string or binary into an M value. A JSON object becomes a record. A JSON array becomes a list. JSON null becomes M null. JSON numbers, strings, booleans map directly.	
Example	<code>Json.Document(Web.Contents("https://api.example.com/user/1"))</code>	
Result	<code>[id=1, name="Alice", email="alice@example.com", ...]</code> (a record)	

NOTE

`Json.Document` returns different M types depending on the root JSON structure: object → record, array → list, scalar → that scalar type. Always inspect the raw response in a staging query before building transformation steps.

Navigating JSON structures — the three shapes every API returns:

```
// Shape 1: Root object with a data array (most common REST API pattern)
// JSON: { "total": 250, "page": 1, "data": [ {...}, {...}, ... ] }
let
Raw = Json.Document( Web.Contents("https://api.example.com/customers") ),
Total = Raw[total], // number
Items = Raw[data], // list of records
Table = Table.FromRecords(Items)
in
Table

// Shape 2: Root array
// JSON: [ {...}, {...}, ... ]
let
Raw = Json.Document( Web.Contents("https://api.example.com/items") ),
// Raw is already a list
Table = Table.FromRecords(Raw)
in
Table

// Shape 3: Deeply nested – JSON: { "results": { "orders": [ {...} ] } }
let
Raw = Json.Document( Web.Contents("https://api.example.com/report") ),
```

```

Orders = Raw[results][orders], // navigate with chained field access
Table = Table.FromRecords(Orders)
in
Table

// Expanding nested record fields inside a table column
// Column [address] contains records: { "city": "London", "postcode": "EC1A" }
Table.ExpandRecordColumn(Source, "address", {"city","postcode"}, {"City","Postcode"})

// Expanding nested list column – each cell is a list of tag strings
Table.ExpandListColumn(Source, "tags") // one row per tag

```

Syntax	Xml.Document(contents as any) as record	
Parameter	Type	Description
contents	binary or text	The XML content — typically the result of Web.Contents.
Returns	Parses XML into a nested record structure. The root element becomes a record. Child elements are nested records or lists. Attributes are available in a sub-record.	
Example	Xml.Document(Web.Contents("https://api.example.com/feed.xml"))	
Result	[Name="feed", Attributes=[...], Value=[...]] (nested record)	

NOTE

XML parsing in M is verbose — use Xml.Tables when the source is a regular tabular XML or SOAP response. Xml.Document gives you the full tree for navigating irregular XML.

```

// Xml.Tables – converts regular XML tables directly (simpler than Xml.Document)
Xml.Tables( Web.Contents("https://api.example.com/data.xml") )
// Returns a list of tables, one per top-level repeated element

// Navigating Xml.Document manually
let
Raw = Xml.Document( Web.Contents("https://api.example.com/orders.xml") ),
// Access child elements: Raw[Value] contains child records
Items= Raw[Value]{0}[Value], // first child's children
Table= Table.FromRecords(
List.Transform(Items,
each [ OrderID=[Attributes][id], Amount=[Value]{0}[Value] ]))
in
Table

```

LESSON 3

Authentication Patterns

After completing this lesson, you will be able to:

- Pass API keys via headers and query parameters.
- Implement Basic authentication with Base64-encoded credentials.
- Pass OAuth 2.0 Bearer tokens in the Authorization header.
- Store credentials securely in Power BI Service without hardcoding them in M.

API Key Authentication

```
// Pattern 1: API key in Authorization header
Web.Contents("https://api.example.com/data", [
  Headers = [ #"Authorization" = "ApiKey " & ApiKey ]
])

// Pattern 2: API key as query parameter (less secure – key appears in URL logs)
Web.Contents("https://api.example.com/data", [
  Query = [ api_key = ApiKey, format = "json" ]
])

// Pattern 3: Custom header name (varies by API provider)
Web.Contents("https://api.example.com/data", [
  Headers = [
    #"X-API-Key" = ApiKey,
    #"X-Client-ID" = ClientID,
    #"Accept" = "application/json"
  ]
])

// Where does ApiKey come from?
// Option A: Power BI parameter (Data > Parameters – prompts at load time)
// Option B: A config table loaded from a secured Excel/SharePoint file
// NEVER hardcode credentials in M source code – they appear in PBIX file plaintext
```

Basic Authentication

```
// Basic Auth: Base64(username:password) in Authorization header
let
  Credentials = "myuser" & ":" & "mypassword",
  Encoded = Binary.ToText(Text.ToBinary(Credentials), BinaryEncoding.Base64),
  AuthHeader = "Basic " & Encoded,
  Response = Web.Contents("https://api.example.com/data", [
    Headers = [ #"Authorization" = AuthHeader ]
  ])
in
  Json.Document(Response)

// Best practice: load credentials from a protected parameter or config table
// NEVER hardcode username/password – use Power BI Service credential manager instead
// In Power BI Service: Dataset > Settings > Data source credentials > Edit credentials
```

OAuth 2.0 Bearer Token

```
// Step 1: Obtain a Bearer token (Client Credentials grant)
let
```

```
TokenResponse = Json.Document(
Web.Contents("https://login.example.com/oauth/token", [
Headers = [ #"Content-Type" = "application/x-www-form-urlencoded" ],
Content = Text.ToBinary(
"grant_type=client_credentials"
& "&client_id=" & ClientID
& "&client_secret=" & ClientSecret
& "&scope=read:data"
)
])
),
AccessToken = TokenResponse[access_token]
in
AccessToken

// Step 2: Use the token in subsequent API calls
let
Token = BearerTokenQuery, // reference the token query above
Response = Web.Contents("https://api.example.com/data", [
Headers = [ #"Authorization" = "Bearer " & Token ]
])
in
Json.Document(Response)
```

WARNING

OAuth tokens expire — typically in 1 hour. The token query must run before every data query. In Power BI Service, tokens obtained in M are refreshed on every scheduled refresh. Structure your queries so the token query is a dependency of the data queries, not a separate standalone query.

LESSON 4

Pagination

After completing this lesson, you will be able to:

- Implement offset/limit pagination with `List.Generate`.
- Implement page-number pagination with a recursive or generative approach.
- Handle cursor-based and next-link pagination.
- Choose the correct pagination pattern for any REST API.

Pagination is required whenever an API returns data in chunks. The three most common styles are offset/limit (skip N, take M), page number (page=1, page=2...), and cursor/next-link (a token or URL in the response points to the next page). All three can be implemented with `List.Generate`.

List.Generate — The Universal Pagination Engine

```
// List.Generate syntax:
// List.Generate( init, condition, next, optional selector )
// init : () => initialState – starting state record
// condition : (state) => logical – continue while true
// next : (state) => nextState – advance to next state
// selector : (state) => value – extract value from state (optional)

// Pattern 1: Offset / Limit pagination
// API: GET /orders?offset=0&limit=100 → { data:[...], total:500 }
let
    PageSize = 100,
    GetPage = (offset as number) =>
        Json.Document( Web.Contents("https://api.example.com", [
            RelativePath = "orders",
            Query = [ offset=Text.From(offset), limit=Text.From(PageSize) ]
        ])),
    AllPages = List.Generate(
        () => [ offset=0, data=GetPage(0) ],
        (s) => not List.IsEmpty(s[data][data]),
        (s) => [ offset = s[offset]+PageSize, data = GetPage(s[offset]+PageSize) ],
        (s) => s[data][data]
    ),
    Combined = Table.FromRecords( List.Combine(AllPages) )
in
    Combined

// Pattern 2: Page-number pagination
// API: GET /items?page=1&per_page=50 → { items:[...], has_more: true/false }
let
    PerPage = 50,
    GetPage = (page as number) =>
        Json.Document( Web.Contents("https://api.example.com", [
            RelativePath = "items",
            Query = [ page=Text.From(page), per_page=Text.From(PerPage) ]
        ])),
    AllPages = List.Generate(
        () => [ page=1, resp=GetPage(1) ],
        (s) => s[resp][has_more] = true or s[page] = 1,
        (s) => [ page=s[page]+1, resp=GetPage(s[page]+1) ],
        (s) => s[resp][items]
    )
```

```
),
Result = Table.FromRecords( List.Combine(AllPages) )
in
Result

// Pattern 3: Next-link / cursor pagination
// API response contains: { "data":[...], "next": "https://api.example.com/items?cursor=abc123" }
let
FirstUrl = "https://api.example.com/items",
GetPage = (url as text) => Json.Document( Web.Contents(url) ),
AllPages = List.Generate(
() => [ url=FirstUrl, resp=GetPage(FirstUrl) ],
(s) => s[url] <> null,
(s) => let nextUrl = try s[resp][next] otherwise null
in [ url=nextUrl, resp=if nextUrl=null then null else GetPage(nextUrl) ],
(s) => s[resp][data]
),
Result = Table.FromRecords( List.Combine( List.RemoveLastN(AllPages,1) ) )
in
Result
```

TIP

Always add a safety limit to List.Generate pagination: change the condition to '...and s[page] <= 200' to prevent infinite loops if the API's has_more flag is always true due to a bug. Without a limit, a misbehaving API will run forever and consume all available memory.

LESSON 5

Dynamic URL and Parameter Construction

After completing this lesson, you will be able to:

- Build URL query strings safely with `Uri.BuildQueryString`.
- Drive API requests from a date range computed at refresh time.
- Construct multi-segment endpoint URLs from config records.
- Loop over a list of resource IDs to fetch multiple endpoints in parallel.

Syntax	<code>Uri.BuildQueryString(parameters as record) as text</code>	
Parameter	Type	Description
parameters	record	A record whose field names become parameter keys and field values become URL-encoded parameter values.
Returns	Returns a URL-encoded query string (without the leading '?'). All values are properly URL-encoded, handling spaces, special characters, etc.	
Example	<code>Uri.BuildQueryString([start="2024-01-01", end="2024-12-31", format="json"])</code>	
Result	<code>"start=2024-01-01&end;=2024-12-31&format;=json"</code>	

NOTE

Use this function when you need a query string as text (e.g. in a POST body). For query parameters in `Web.Contents`, use the `Query` field in the options record — it calls `Uri.BuildQueryString` internally.

Date-driven API requests — computing date ranges at refresh time:

```
// Compute a rolling 90-day window dynamically – no hardcoded dates
let
    Today = Date.From(DateTime.LocalNow()),
    StartDate = Date.AddDays(Today, -90),
    EndDate = Today,
    DateFmt = (d) => Date.ToText(d, "yyyy-MM-dd"),

    Response = Json.Document(
        Web.Contents("https://api.example.com", [
            RelativePath = "transactions",
            Query = [
                from = DateFmt(StartDate),
                to = DateFmt(EndDate),
                limit = "500"
            ]
        ])
    )
in
    Table.FromRecords(Response[data])
```

Fetching multiple endpoints from a config table — parallel requests:

```
// Config table in Excel: Endpoint | Entity | Active
// e.g. /api/customers | Customer | true
// /api/products | Product | true
// /api/orders | Order | true

let
Config = Excel.Workbook(File.Contents("APIConfig.xlsx")){[Name="Endpoints"]}[Data],
Active = Table.SelectRows(Config, each [Active] = true),
Endpoints= Active[Endpoint],

// Fetch all active endpoints; return list of tables
AllData = List.Transform(Endpoints, (ep) =>
Table.FromRecords(
Json.Document(
Web.Contents("https://api.example.com", [ RelativePath = ep ])
)[data]
),
),
Combined = Table.Combine(AllData)
in
Combined
```

TIP

List.Transform over a list of URLs is M's closest equivalent to parallel requests. Power Query evaluates each Web.Contents call and may parallelise them depending on the connector and service configuration. This is faster than a recursive loop for a known, bounded list of endpoints.

LESSON 6

Robustness and Error Handling

After completing this lesson, you will be able to:

- Inspect HTTP status codes using `ManualStatusHandling` and `Value.Metadata`.
- Implement retry logic for 429 Too Many Requests responses.
- Validate API responses before transformation to prevent silent data issues.
- Build incremental load patterns to avoid re-fetching historical data.

HTTP Status Code Handling and Retry Logic

```
// Full defensive request with status inspection and retry
let
FetchWithRetry = (url as text, relPath as text, queryParams as record,
attempt as number) as any =>
let
Response = Web.Contents(url, [
RelativePath = relPath,
Query = queryParams,
ManualStatusHandling = {400,401,403,404,429,500,502,503},
IsRetry = attempt > 1
]),
Status = Value.Metadata(Response)[Response.Status]
in
if Status = 200 then Json.Document(Response)
else if Status = 429 then
// Rate limited – wait and retry (up to 3 attempts)
if attempt >= 3
then error Error.Record("RateLimit",
"Rate limited after 3 attempts", [Status=Status])
else @FetchWithRetry(url, relPath,
Record.AddField(queryParams, "_ts", Text.From(attempt+1)),
attempt+1)
else if Status = 404 then null // resource not found – return null
else if Status = 401 then
error Error.Record("Auth","Unauthorised – check API credentials",[Status=Status])
else
error Error.Record("HttpError",
"Unexpected HTTP " & Text.From(Status), [Status=Status])
in
FetchWithRetry("https://api.example.com", "orders", [limit="100"], 1)
```

Response Validation

```
// Validate API response structure before transformation
let
Raw = Json.Document( Web.Contents("https://api.example.com/data") ),

// Check expected fields exist in the response
RequiredFields = {"data","total","page"},
Missing = List.Difference(RequiredFields, Record.FieldNames(Raw)),

Validated =
if not List.IsEmpty(Missing)
then error Error.Record("Schema",
"API response missing fields: " & Text.Combine(Missing, ", "), Raw)
```

```
else Raw,

// Check data is a list (not null or wrong type)
DataList =
if Value.Is(Validated[data], type list)
then Validated[data]
else error Error.Record("Schema","Expected list at [data], got: " &
Text.From(Value.Type(Validated[data])), null),

Result = Table.FromRecords(DataList)
in
Result
```

Incremental Load Pattern

```
// Load only records created/modified since the last known watermark
// Watermark stored in a Power BI parameter: LastLoadedDate (type date)
let
Watermark = DateTime.ToText(
DateTime.From(LastLoadedDate), "yyyy-MM-ddTHH:mm:ssZ"),
Response = Json.Document(
Web.Contents("https://api.example.com", [
RelativePath = "orders",
Query = [
modified_since = Watermark,
limit = "1000",
sort = "modified_at:asc"
]
])),
NewRecords = Table.FromRecords(Response[data]),
// Combine with existing data (using Table.Combine and Distinct on key)
Combined = Table.Distinct(
Table.Combine({HistoricalData, NewRecords}),
"OrderID")
in
Combined
```

LESSON 7

Real-World API Patterns for PL-300

After completing this lesson, you will be able to:

- Build a complete production REST connector with auth, pagination, and error handling.
- Drive a multi-endpoint connector from a config table.
- Implement delta/incremental refresh via API watermark.
- Fetch a list of resource IDs in parallel with List.Transform.

Pattern 1: Complete Production REST Connector

[illegible]

Pattern 2: Multi-Endpoint Connector from Config

```
// Config record defines endpoints, entity names, and key columns
let
Config = [
```

```
Customers = [ Path="customers", Key="id", Type=type text ],
Orders = [ Path="orders", Key="order_id", Type=Int64.Type ],
Products = [ Path="products", Key="sku", Type=type text ]
],
ApiBase = "https://api.example.com",
AuthHdr = [ #"Authorization" = "Bearer " & BearerToken ],

FetchEntity = (cfg as record) as table =>
let
Raw = Json.Document(Web.Contents(ApiBase,
[ RelativePath=cfg[Path], Headers=AuthHdr ])),
Tbl = Table.FromRecords(Raw[data]),
Typed = Table.TransformColumnTypes(Tbl, {{cfg[Key], cfg[Type]}})
in Typed,

// Fetch all three entities
CustomerTable = FetchEntity(Config[Customers]),
OrderTable = FetchEntity(Config[Orders]),
ProductTable = FetchEntity(Config[Products])
in
{ CustomerTable, OrderTable, ProductTable }
```

DEMONSTRATION 1

GitHub REST API — Paginated Issues

Scenario: Fetch all open issues from a public GitHub repository using the GitHub REST API. The API returns 30 issues per page by default. Build a paginated connector that retrieves all pages and produces a flat typed table.

Demonstration Steps

1	Open Power BI Desktop. Create a new query. In the Advanced Editor, make a single Web.Contents call to <code>https://api.github.com/repos/microsoft/vscode/issues?per_page=5</code> . Show the raw binary result, then pipe to <code>Json.Document</code> . Show the list of records.
2	Convert the list to a table with <code>Table.FromRecords</code> . Identify the key columns: <code>number</code> , <code>title</code> , <code>state</code> , <code>created_at</code> , <code>user</code> (nested record), <code>labels</code> (list of records).
3	Add pagination with <code>List.Generate</code> using page-number pagination. The GitHub API uses a <code>Link</code> response header — show the simpler page-number approach first. Set <code>per_page=100</code> and a safety limit of 20 pages (2000 issues max).
4	Expand the nested user record to extract login (username). Expand the labels list column to get one row per label, then re-group to get a comma-separated label string per issue.
5	Apply <code>Table.TransformColumnTypes</code> : <code>number</code> → <code>Int64</code> , <code>created_at</code> → <code>datetime</code> , <code>state</code> → <code>text</code> , <code>title</code> → <code>text</code> .
6	Add <code>RelativePath</code> and <code>Headers</code> to the <code>Web.Contents</code> call. Show that the base URL <code>https://api.github.com</code> is now static — this enables Power BI Service caching.
7	Demonstrate <code>ManualStatusHandling</code> : change the repo to a non-existent name. Show the query returns null instead of crashing. Restore the correct repo name.

RESULT

A fully paginated, typed GitHub issues table with columns: `IssueNumber`, `Title`, `State`, `CreatedAt`, `Author`, `Labels`. All pages fetched. The connector handles the 404 case gracefully. `RelativePath` ensures the base URL is static for Power BI Service compatibility.

DEMONSTRATION 2

Multi-Endpoint CRM Connector

Scenario: A client uses a CRM with a REST API exposing three entities: Contacts, Deals, and Activities. Build a single config-driven connector that fetches all three, applies shared transformation logic, and loads them as three separate tables.

Demonstration Steps

1	Create the APIConfig.xlsx file with an Endpoints sheet: columns Endpoint, Entity, PageSize, Active.
2	Load the config table into Power Query. Filter to Active=true. Show how new endpoints can be added to the config without touching M code.
3	Define a reusable FetchEntity function in a blank query. It accepts an endpoint config record and returns a fully paginated, typed table.
4	Apply FetchEntity to each active config row using List.Transform. Show that all three entities are fetched in a single let...in expression.
5	For Contacts: expand the nested address record to City, Country.
6	For Deals: expand the nested owner record to OwnerName. Apply currency formatting.
7	For Activities: expand the list-valued contacts field — one activity row per linked contact.
8	Load all three tables into the Power BI model. Build a simple relationship diagram showing Deals ← Activities and Contacts ← Activities. Run a measure: total deal value by owner.

RESULT

Three typed entity tables loaded into the Power BI model from a single config-driven connector. Adding a new entity requires only a new row in APIConfig.xlsx — no M code changes. All three tables share the same FetchEntity function.

LAB 8

API Integration and Web Connectors

In this lab you build production-grade API connectors using the public GitHub REST API (no credentials required for public repos). All exercises use `Web.Contents`, `Json.Document`, pagination, authentication, and error handling.

Lab Scenario

You are a Power BI developer at a software company. Management wants a live dashboard showing open GitHub issues, pull requests, and contributor activity across three repositories. You must build the M connectors from scratch.

NOTE

Use the GitHub public REST API: <https://api.github.com>. All read operations on public repositories are available without authentication. Rate limit: 60 requests/hour unauthenticated, 5000/hour with a Personal Access Token.

Exercise 1: Basic REST Requests and JSON Parsing (Estimated time: 20 minutes)

- Write a `Web.Contents` call to <https://api.github.com/repos/microsoft/powerbi-desktop-samples/issues>. Pipe to `Json.Document`. Inspect the raw structure — identify all nested records and lists.
- Convert the response list to a table. Expand the user record to extract login. Expand the labels list — one row per label.
- Apply `Table.SelectColumns` to keep: number, title, state, created_at, user.login, labels.
- Apply `Table.TransformColumnTypes`: number→Int64, created_at→type datetime, all text columns→type text.
- Add a DaysOpen column: `Duration.Days(DateTime.LocalNow() - DateTime.From([created_at]))`.
- STRETCH: Also fetch the pull_requests endpoint and `Table.Combine` with issues. Add a Type column (Issue vs PR) before combining.

RESULT

A typed issues table with IssueNumber, Title, State, CreatedAt, Author, Label, DaysOpen. Date and integer columns correctly typed.

Exercise 2: Authentication and Secure Refresh (Estimated time: 20 minutes)

- Create a Power BI parameter named `GitHubToken` of type Text. Leave it empty for now.
- Modify the `Web.Contents` call to include Headers: if `GitHubToken = ""` then `[]` else `[#"Authorization" = "Bearer " & GitHubToken]`. This makes auth optional.
- Verify the query works without a token (unauthenticated, 60 req/hour limit).
- If you have a GitHub Personal Access Token: populate the parameter and verify the rate limit header in the response changes to 5000.
- Add `ManualStatusHandling = {401, 403, 429}` to the `Web.Contents` call. Add status inspection: return a descriptive error for 401 and 403, and null with a warning message for 429.
- STRETCH: Move the authentication header construction into a `FunctionLibrary[GitHubAuth]` function (from Module 7 Lab) that accepts an optional token.

RESULT

The connector works anonymously and with a token. HTTP errors return descriptive messages instead of cryptic binary errors. The token parameter can be populated in Power BI Service credential manager.

Exercise 3: Pagination Patterns (Estimated time: 25 minutes)

- The GitHub issues endpoint returns at most 100 items per page. Add `?per_page=10&page;=1` to your request. Observe the response.
- Implement page-number pagination with `List.Generate`: fetch pages 1 to N where N is determined by the response (stop when fewer than `per_page` items returned).
- Add a safety limit of 30 pages (3000 items max) to the `List.Generate` condition.
- `Table.Combine` all pages. Verify the total row count matches the repository's issue count.
- Implement cursor pagination: use the `Link` response header from GitHub (`Value.Metadata(response)[Headers][#"Link"]`) to extract the next-page URL.
- Compare the page-number and cursor approaches — which is more robust for this API?
- STRETCH: Apply the same pagination pattern to fetch all pull requests and all commits from three repositories. Show a combined summary table.

RESULT

A fully paginated issues dataset with all pages fetched. Safety limit prevents infinite loops. Both pagination patterns implemented and compared.

Exercise 4: Dynamic URLs and Production Connector (Estimated time: 25 minutes)

- Load `APIConfig.xlsx`. Create a config table with three GitHub repos: `microsoft/powerbi-desktop-samples`, `microsoft/vscode`, and `dotnet/runtime`.
- Build a `FetchRepoIssues` function that takes a repo path (e.g. `'microsoft/vscode'`) and returns a paginated typed issues table. Use `RelativePath` to keep the base URL static.
- Apply `FetchRepoIssues` to all three repos using `List.Transform`. Add a `RepoName` column to each table before combining with `Table.Combine`.
- Add date-driven filtering: only fetch issues created in the last 90 days using the `since=` query parameter with `Date.AddDays(Date.From(DateTime.LocalNow()), -90)`.
- Wrap the entire connector in `try...otherwise` to return an empty typed table if any repo is unreachable or returns a non-200 response.
- Load the combined table into Power BI. Build a bar chart of issue count by repository and a line chart of issues opened per week.
- STRETCH: Add a fourth endpoint — contributor statistics — and join it to the issues table to show which contributors are most active per repository.

RESULT

A config-driven multi-repo GitHub connector fetching issues from three repositories. Date-filtered to 90 days. Error-tolerant per repo. Loaded into Power BI with two working visualisations.

Module Review and Takeaways

Review Questions

Why should query parameters always be passed in the Query record rather than concatenated into the URL string?

Answer: The Query record handles URL encoding automatically — spaces, ampersands, and special characters are correctly escaped. Concatenation breaks on values containing these characters and is a security risk. Additionally, the Query record keeps the base URL static, which is required for Power BI Service to correctly cache and validate the data source.

What is the purpose of RelativePath in Web.Contents?

Answer: RelativePath separates the dynamic path segments from the static base URL. Power BI Service validates and caches data sources by base URL. If dynamic values (page numbers, resource IDs) are embedded in the base URL string, the refresh validation check fails or creates unintended separate cached entries. RelativePath keeps the base URL fixed while allowing dynamic path construction.

How does ManualStatusHandling change the behaviour of Web.Contents?

Answer: By default, Web.Contents raises an M-level error for any non-2xx HTTP response — the error message is often unhelpful. ManualStatusHandling accepts a list of status codes that should NOT automatically raise errors. The response binary is returned regardless, and the HTTP status is available in Value.Metadata(response)[Response.Status]. This lets you inspect the status code and handle each case explicitly in M.

What are the three pagination styles and which M function implements all three?

Answer: Offset/limit (skip N, take M), page-number (page=1, page=2...), and cursor/next-link (a token or URL in the response points to the next page). List.Generate implements all three by maintaining state between iterations: the state record holds the current page/offset/cursor and the fetched response. The condition stops when the response indicates no more data.

How should API credentials be stored in a production Power BI model?

Answer: Never hardcode credentials in M source code — they are stored as plaintext in the PBIX file. Use Power BI parameters for the credential values, and populate them via Data source credentials in Power BI Service (Dataset > Settings > Data source credentials). For OAuth, the token query runs on every refresh and obtains a fresh token — the client secret should be in a parameter, not hardcoded.

What is the difference between Json.Document returning a record vs returning a list?

Answer: If the root of the JSON response is an object (starts with '{'), Json.Document returns a record. If the root is an array (starts with '['), it returns a list. Most REST APIs wrap their data array inside an object with metadata fields (total, page, next) — you access the data array with Raw[data]. APIs that return a root array are rarer but simpler — the result is already a list you can pass directly to Table.FromRecords.

Why should you add a safety limit to List.Generate pagination?

Answer: If the API's has_more flag has a bug and is always true, or if a cursor never advances, List.Generate will loop indefinitely, consuming memory until the refresh crashes. A safety limit (e.g. and s[page] <= 200) bounds the maximum pages fetched. Choose a limit that is comfortably above the expected maximum but prevents runaway queries.

What is incremental load in the context of API connectors and how is it implemented in M?

Answer: Incremental load fetches only records created or modified since the last load — identified by a watermark (typically the most recent date loaded). In M, the watermark is stored in a Power BI parameter (LastLoadedDate). The API request includes a modified_since or since parameter set to the watermark date. After loading, the new maximum date becomes the next watermark. This dramatically reduces API calls and refresh time for large datasets.

Best Practices

- Always use the Query record in Web.Contents options for query parameters — never string concatenation. The Query record URL-encodes values correctly and keeps the base URL static for Power BI Service.
- Use RelativePath for dynamic URL path segments. The base URL must be a string literal or a parameter — it must not be computed dynamically, or Power BI Service refresh validation fails.
- Always add ManualStatusHandling to production Web.Contents calls. Handle 401, 403, 429, and 500 explicitly. A 429 should retry; a 401 should raise an authentication error.
- Add a safety limit to all List.Generate pagination loops. Without it, a misbehaving API will loop indefinitely and crash the refresh.
- Store API credentials in Power BI parameters and configure them via the Service credential manager — never hardcode them in M source code.
- Always validate the API response structure before transformation. Check that expected fields exist and that the data field is a list before calling Table.FromRecords.
- Use Table.Buffer on the token query result when it is referenced by multiple data queries. Without buffering, each query independently calls the token endpoint, wasting requests.
- For paginated APIs, always inspect the raw first-page response in a staging query before building pagination logic. The page structure (field names, stop condition) varies by API and must be understood before writing List.Generate.
- Use incremental load patterns for any API with more than a few thousand records. Full refresh of large APIs is slow, consumes API quota, and may hit rate limits.
- Test all error paths: non-existent resource (404), expired token (401), and rate limit (429). Use ManualStatusHandling to inject these conditions and verify your error handling returns sensible values.

Common Issues and Solutions

'Formula.Firewall' error when combining Web.Contents queries

Cause: Power BI's data privacy firewall prevents combining data from different data sources in a single query when privacy levels are not configured. This affects queries that reference both a web source and a local file (e.g. config table).

Solution: Set privacy levels for all data sources in Power BI Desktop: File > Options > Privacy > Ignore Privacy Levels. For production models, set appropriate privacy levels (Public/Organisational/Private) on each source in Power BI Service.

Web.Contents works in Desktop but fails in Power BI Service refresh

Cause: The most common cause: dynamic values (page numbers, dates) are concatenated into the URL string rather than passed as RelativePath or Query. Power BI Service validates that the URL base hasn't changed between design and refresh.

Solution: Refactor Web.Contents to use a static literal base URL with RelativePath and Query for all dynamic values. Test by publishing a PBIX and manually triggering a refresh.

Json.Document raises an error on a valid API response

Cause: The response contains a Byte Order Mark (BOM) or the Content-Type header specifies an unexpected encoding, causing binary parsing to fail.

Solution: Pipe the response through `Binary.Buffer` first, then to `Json.Document`. Or specify encoding explicitly: `Json.Document(response, 1200)` for UTF-16.

Rate limit hit during List.Generate pagination

Cause: Fetching hundreds of pages in rapid succession triggers the API's rate limiter, returning 429 Too Many Requests mid-pagination.

Solution: Add `ManualStatusHandling={429}` inside `FetchPage`. When `status=429`, read the `Retry-After` header from `Value.Metadata(response)[Headers]` and add a `Function.InvokeAfter` delay. Alternatively, reduce page size to make more, smaller requests over a longer period.

Nested record expansion creates unexpected column names

Cause: `Table.ExpandRecordColumn` uses the original field names from the record as column names. If the API uses `snake_case` (e.g. `first_name`), the expanded columns also use `snake_case`.

Solution: Always specify the fourth parameter of `Table.ExpandRecordColumn` to provide explicit renamed column names: `Table.ExpandRecordColumn(t, "user", {"first_name"}, {"FirstName"})`.

Key Functions — Module 8 Quick Reference

Function	Category	Key Behaviour
<code>Web.Contents(url, opts)</code>	HTTP	Make HTTP requests; returns binary response
<code>Json.Document(binary)</code>	Parse	Parse JSON binary → M record/list/value
<code>Xml.Document(binary)</code>	Parse	Parse XML binary → nested M record
<code>Xml.Tables(binary)</code>	Parse	Parse tabular XML → list of tables
<code>Value.Metadata(binary)[Response.Status]</code>	HTTP	Get HTTP status code (ManualStatusHandling)
<code>Uri.BuildQueryString(record)</code>	URL	Build URL-encoded query string from record
<code>Binary.ToText(binary, BinaryEncoding.Base64)</code>	Auth	Base64-encode for Basic Auth
<code>Text.ToBinary(text)</code>	Auth	Convert text to binary for Content field
<code>List.Generate(init, cond, next, sel)</code>	Pagination	Universal pagination engine
<code>Table.FromRecords(list)</code>	Parse	Convert list of records to table
<code>Table.ExpandRecordColumn(t, col, fields, rename)</code>	Parse	Flatten nested [Record] column
<code>Table.ExpandListColumn(t, col)</code>	Parse	Explode nested [List] column into rows
<code>Date.ToText(date, format)</code>	URL	Format date for API query parameters
<code>DateTime.LocalNow()</code>	Incremental	Current datetime for watermark / delta load
<code>Function.InvokeAfter(fn, delay)</code>	Rate Limit	Delay execution for rate limit handling

Module Summary

- `Web.Contents` is the foundation of all API work in M. Its options record — `Query`, `Headers`, `RelativePath`, `ManualStatusHandling` — controls every aspect of the HTTP request. Always use `RelativePath` and `Query` for dynamic values; never concatenate into the URL string.
- `Json.Document` parses JSON binary into M values. A JSON object becomes a record; an array becomes a list. Navigate nested structures with chained field access and expand with `Table.ExpandRecordColumn` and `Table.ExpandListColumn`.
- Authentication patterns: API Key (Header or Query), Basic Auth (Base64 encoded), OAuth 2.0 (Bearer token from a token endpoint). Always store credentials in Power BI parameters — never hardcode them in M.
- `List.Generate` is the universal pagination engine. It maintains state across pages, supports all three pagination styles (offset, page-number, cursor/next-link), and must always include a safety limit to prevent infinite loops.
- Dynamic URL construction: use the `Query` record for parameters, `RelativePath` for path segments, `Uri.BuildQueryString` for POST bodies. Drive requests from config tables to add new endpoints without code changes.
- Production connectors require: `ManualStatusHandling` for HTTP errors, response validation for schema changes, retry logic for 429 rate limits, and incremental load patterns for large datasets.
- Four production patterns: complete paginated REST connector, multi-endpoint config-driven connector, delta/incremental refresh, and parallel requests with `List.Transform` over a list of resource IDs.

NOTE

Next Module — Module 9: Query Folding and Performance. Covers how M expressions are translated to native source queries, which operations fold and which break folding, how to diagnose folding in Power BI Desktop, and the architectural patterns that maximise query folding for SQL Server, SharePoint, OData, and other foldable connectors.